

Suivi du projet

Code

Architecture

Tour du NO : Pour implémenter la tour du NO, nous avons trois classes dans le modèle qui représentent une salle, un étage et la tour, ainsi que trois contrôleurs respectifs (RoomManager, FloorManager et TowerManager). Un GameController a été ajouté en plus du BattleController qui était déjà présent, afin de permettre de contrôler la tour et la connecter avec la vue. Cette architecture devrait, dans le futur, permettre d'assez facilement apporter des changements à la structure de la tour, lors des histoires éventuelles qui le demandent.

Avancée des histoires

Tâches accomplies

Tâches inachevées lors de l'itération précédente :

- Histoire 5 - Contrôle des actions : Afficher la fenêtre de Switch de Bugémon après la mort du Bugémon actif
- Histoire 6 - Calcul de dégâts : Afficher les messages d'efficacité dans les logs
- Histoire 7 - Montée d'expérience : Générer et afficher les récompenses disponibles à la montée de niveau

Tâches ajoutées lors de cette itération :

- Histoire 9 - Tour du NO
- Histoire Complémentaire - Multijoueur : Créer une connexion Client - Server et afficher dans le terminal lorsque le client se connecte. La création d'un client et d'un serveur se fait donc à part du jeu (celui-ci ne se lance pas lorsque les arguments "--server" ou "--client" sont donné à l'exécution

Tâches non accomplies

Seul le refactor du code en MVC n'a pas été totalement fini.

Refactor

MVC

Nous avons reçu comme instruction de notre coach de refactorer le code pour mieux respecter l'architecture MVC. Nous avons entamé ce travail mais n'avons pas eu le temps d'en venir à bout.

Les responsabilités de la classe Battle du modèle ont été dispersées dans d'autres classes afin d'alléger cette première. Ceci nous permettra d'y voir plus clair dans le code et de plus facilement déplacer d'autres responsabilités en lien avec les combats.

Il nous reste à déplacer la logique d'application des actions des joueurs du contrôleur vers le modèle. Nous n'avons pas encore retiré l'interface vide Action car nous prévoyons de lui attribuer des méthodes lorsque nous entamerons cette tâche.

Nous avons planifié la restructuration des rôles entre la Vue et le Contrôleur et avons commencé à implémenter cette structure. A terme, le but étant que la Vue communique au Contrôleur les actions choisies par le joueur via un protocole de communication. Le contrôleur va alors répondre avec les nouvelles informations / la nouvelle fenêtre à afficher. Nous utilisons des classes qui héritent toutes d'une interface Message Serializable ainsi que des DTO comme moyen de communication, ce qui rendra le passage vers une communication via des sockets plus facile.

Repository

Le Repository a été simplifié. Il présentait un excès de généricité qui rendait l'architecture de la couche difficile à comprendre.

Les anciennes classes que le Repository avait pour but de remplacer ont été entièrement retirées du code. Le chargement de données de test se fait maintenant avec des fichiers JSON situés dans le dossier "src/test/resources/mock/", et les classes statiques contenant les données statiques ont été remplacées par des Repositories et Services à injecter.

Il reste encore une classe statique ServiceLoader qui charge les services car nous n'avons pas encore défini l'endroit du code où les services devraient se faire charger avant d'être injectés.